Tool-Call F1: Precision, Recall, Micro vs Macro, Worked Out by Hand

How to score whether an agent called the *right* tools —
and why averaging that score two different ways can hide a critical failure.

What this document does. It takes the *Refund Resolver* from Module 3.1 — the support agent whose gold tool path is `crm_lookup` → `shipping_status` → `refund` — and treats its tool calls as a set-retrieval problem. We fix a tiny eval set, count for each tool how often it was correctly called (TP), wrongly called (FP), and missed (FN), and from those nine integers compute precision, recall, and F1 by hand. We then average across the three tools *two* ways — **micro** (pool first, divide once) and **macro** (mean of per-tool F1) — and watch them disagree because the critical `refund` tool is rare. Every number below is reproduced by the Python program in Appendix A, so the arithmetic is exact. The module quotes a single-trajectory headline of `F1 = 0.86`; we rebuild that figure and then scale the idea up to a whole suite.

Where this sits. This is **Topic 1 of Module 3.1** — the math behind Pillar 2, *Tool Accuracy*, and its headline metric `tool_F1`. Its companions: Topic 2 of this module (percentiles for step budgets and cost — the distributional cousin of this per-tool view) and Module 3.6 (LLM-as-judge, for the quality that F1 cannot see).

Concepts covered, in order: tool calls as a confusion of expected vs. actual · per-tool TP / FP / FN · precision $= \frac{TP}{TP+FP}$, recall $= \frac{TP}{TP+FN}$ · F1 as the harmonic mean · the identity F1 $= \frac{2TP}{2TP+FP+FN}$ · micro averaging (pool, weight by frequency) · macro averaging (mean per class) · why they split under class imbalance · which one to put in your CI gate.

Internal learning material. Numbers are reproducible from Appendix A. v1.0

1 The setup: tool calls as a retrieval problem

When an agent finishes a task, the runtime has a log of which tools it called. We compare that *actual* set against the *expected* (gold) set and ask, tool by tool: did the agent get this one right? Three outcomes matter for each tool k:

- **TP** (true positive): the gold path needed tool k and the agent called it.
- **FP** (false positive): the agent called k but the gold path did not need it — an *extra*, like the stray `email_user` in the module's example.
- **FN** (false negative): the gold path needed k but the agent never called it — a *miss*, the dangerous one.

We score not one trajectory but a small *eval suite* of runs of the Refund Resolver, pooling each tool's outcomes across all runs. The three tools and their tallies:

Tool k	TP	FP	FN	support	character
`crm_lookup`	18	2	0	18	frequent, almost always right
`shipping_status`	12	3	5	17	frequent, sometimes skipped
`refund`	2	4	4	6	**rare, and handled badly**
Pool (all tools)	32	9	9	41	—

Notation. *Support* for tool k is the number of times it truly should have been called, $\mathrm{TP}_k + \mathrm{FN}_k$. The pool row sums each column over the three tools. Note the imbalance: `crm_lookup` carries support 18 while `refund`, the tool that actually moves money, has support only 6. That gap is the whole point of this document.

Intuition

Precision answers "*of the tools the agent called, how many should it have?*" — it punishes noise and over-calling. Recall answers "*of the tools it should have called, how many did it?*" — it punishes misses. F1 forces you to be good at *both* at once: you cannot win it by calling everything (high recall, low precision) or by calling almost nothing (high precision, low recall). It is the single number that says "right tools, and only the right tools."

2 Step 1 — precision, recall, and the F1 identity

For a single tool with counts $\mathrm{TP}, \mathrm{FP}, \mathrm{FN}$:

$$P = \frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FP}}, \qquad R = \frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FN}}, \qquad \mathrm{F1} = \frac{2PR}{P+R}.$$

F1 is the *harmonic mean* of P and R. We can rewrite it directly in the counts, which is both faster to compute and reveals its symmetry. Start from the harmonic mean definition:

$$\mathrm{F1} = \frac{2}{\frac{1}{P}+\frac{1}{R}} = \frac{2}{\frac{\mathrm{TP}+\mathrm{FP}}{\mathrm{TP}} + \frac{\mathrm{TP}+\mathrm{FN}}{\mathrm{TP}}} = \frac{2}{\frac{2\,\mathrm{TP}+\mathrm{FP}+\mathrm{FN}}{\mathrm{TP}}}$$
$$= \boxed{\frac{2\,\mathrm{TP}}{2\,\mathrm{TP}+\mathrm{FP}+\mathrm{FN}}} \tag{1}$$

The harmonic mean (not the plain average) is what makes F1 *conservative*: it sits close to the *smaller* of P and R. If either collapses toward 0, so does F1, no matter how good the other is. That is exactly the behaviour you want from a safety metric.

Mini-example — this operation in isolation

The module's headline single trajectory: gold = {`crm_lookup`, `shipping_status`, `refund`}; the agent called those three *plus* an extra `email_user`. So TP = 3 (all three golds hit), FP = 1 (the extra), FN = 0 (nothing missed). Then $P = \frac{3}{4} = 0.75$, $R = \frac{3}{3} = 1.00$, and by Equation (1), $\mathrm{F1} = \frac{2\cdot 3}{2\cdot 3+1+0} = \frac{6}{7} = 0.857$. ✓ That is the module's `F1 = 0.86`, rebuilt from the counts. High recall, lower precision: the agent did everything needed, plus a harmless-looking extra.

3 Step 2 — per-tool scores for the suite

Now apply P, R, and Equation (1) to each of the three tools from the table.

Tool	$P = \frac{\text{TP}}{\text{TP+FP}}$	$R = \frac{\text{TP}}{\text{TP+FN}}$	F1	reading
`crm_lookup`	$\frac{18}{20} = 0.900$	$\frac{18}{18} = 1.000$	0.9474	clean
`shipping_status`	$\frac{12}{15} = 0.800$	$\frac{12}{17} = 0.7059$	0.7500	some misses
`refund`	$\frac{2}{6} = 0.3333$	$\frac{2}{6} = 0.3333$	0.3333	**failing**

Sample check, `shipping_status`: $P = \frac{12}{12+3} = 0.8$, $R = \frac{12}{12+5} = 0.7059$; identity $\frac{2 \cdot 12}{2 \cdot 12+3+5} = \frac{24}{32} = 0.75$. ✓ And for `refund`, $\frac{2 \cdot 2}{2 \cdot 2+4+4} = \frac{4}{12} = 0.3333$. ✓ The story in the numbers: two tools are healthy, but `refund` — the one that issues money — is right only a third of the time. A good aggregate metric must *not* let that vanish.

4 Step 3 — macro averaging: every tool counts the same

The **macro** F1 is the plain mean of the per-tool F1 scores. Each tool gets one vote, regardless of how often it is used:

$$\text{F1}_{\text{macro}} = \frac{1}{K} \sum_{k=1}^{K} \text{F1}_k = \frac{0.9474 + 0.7500 + 0.3333}{3} = \frac{2.0307}{3} = \mathbf{0.6769}.$$

Because `refund` scores a third one of three full votes, its failure drags the macro average down hard. Macro is the voice of the rare class.

> **Mini-example — this operation in isolation**
>
> Macro is an average *of averages*: it first collapses each tool to a single F1, then treats those three numbers as equals. A tool used twice and a tool used two hundred times count the same. That is a feature when each tool is independently important — you do not want a flood of easy `crm_lookup` calls to paper over a broken `refund`.

5 Step 4 — micro averaging: pool first, divide once

The **micro** F1 throws all the counts into one pool and computes a single P, R, F1 from the totals. Using the pool row (TP = 32, FP = 9, FN = 9):

$$P_{\text{micro}} = \frac{32}{32+9} = \frac{32}{41} = 0.7805, \qquad R_{\text{micro}} = \frac{32}{32+9} = \frac{32}{41} = 0.7805,$$

$$\text{F1}_{\text{micro}} = \frac{2 \cdot 32}{2 \cdot 32 + 9 + 9} = \frac{64}{82} = \mathbf{0.7805}.$$

(When the FP and FN totals are equal, micro precision, recall, and F1 all coincide — a useful sanity tell.) Each *call* gets one vote here, so frequent tools dominate. The 30 well-handled `crm_lookup` and `shipping_status` TPs outnumber `refund`'s troubles, so the pool looks healthy at 0.78.

Intuition

Macro weights by **tool**; micro weights by **call**. Macro asks "is each tool, as a capability, working?" Micro asks "across all the tool-using the agent did, what fraction landed?" Neither is wrong — they answer different questions. Trouble starts when you quote one and *think* you measured the other.

6 Step 5 — why they disagree, and which to trust

Lay the two aggregates side by side against the per-tool detail:

	F1	support / weight	what it sees
`crm_lookup`	0.9474	18	—
`shipping_status`	0.7500	17	—
`refund`	0.3333	6	—
Macro (mean of rows)	**0.6769**	each tool = 1	the broken `refund`
Micro (pool of calls)	**0.7805**	each call = 1	the healthy majority

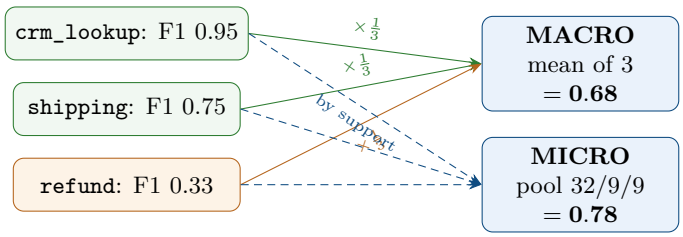
The gap is $0.7805 - 0.6769 = 0.1036$ — over ten F1 points — and it points in a predictable direction: **micro is higher because the failing tool is rare.** Micro lets the abundant, well-handled calls outvote the scarce, critical ones. Macro refuses to, so it sits lower and flashes the warning.

Watch out

If you SLA on **micro** F1 and your most dangerous tool (`refund`, `delete`, `send_payment`) is rare, your dashboard can read "0.78, ship it" while that tool is correct only a third of the time. A micro metric *structurally* hides rare-class failure — the rarer and more critical the tool, the more it hides. Class imbalance is not an edge case in agent tooling; it is the normal case, because the consequential tools fire least often.

So which do you use? Report *both*, and gate on the one that matches what you fear. If every tool must independently work — the usual case when a rare tool is the costly one — gate on **macro** (and, better still, on the *per-tool* F1 of the tools you cannot afford to get wrong). Use micro when you genuinely care about the aggregate hit-rate of all tool-using and all tools carry similar stakes. The module's target of F1 ≥ 0.85 is only safe as a gate if it is a *macro* or per-tool floor; as a micro floor it would have passed this very suite.

7 The two averages, drawn



Macro (solid) gives each tool one-third of a vote, so the orange `refund` pulls the result down to 0.68. Micro (dashed) weights by how many calls each tool made, so the same orange box barely registers and the result floats up to 0.78. *Same data, two stories* — the difference is entirely in the weights.

8 Tool-F1 cheat-sheet

Precision (per tool)	$P_k = \dfrac{\mathrm{TP}_k}{\mathrm{TP}_k + \mathrm{FP}_k}$
Recall (per tool)	$R_k = \dfrac{\mathrm{TP}_k}{\mathrm{TP}_k + \mathrm{FN}_k}$
F1 (harmonic mean)	$\mathrm{F1}_k = \dfrac{2P_kR_k}{P_k + R_k} = \dfrac{2\,\mathrm{TP}_k}{2\,\mathrm{TP}_k + \mathrm{FP}_k + \mathrm{FN}_k}$
Macro F1	$\dfrac{1}{K}\sum_k \mathrm{F1}_k$ (each tool equal)
Micro F1	$\dfrac{2\sum_k \mathrm{TP}_k}{2\sum_k \mathrm{TP}_k + \sum_k \mathrm{FP}_k + \sum_k \mathrm{FN}_k}$ (each call equal)
Imbalance rule	micro > macro ⇒ a rare tool is failing; inspect per-tool
Gate choice	rare-but-critical tools ⇒ gate on macro / per-tool, not micro

9 An honest word about these numbers

The nine counts here (18/2/0, 12/3/5, 2/4/4) are illustrative round figures, chosen so every fraction is followable by hand and so the rare-tool gap is unmistakable — exactly as the gold ReAct document used round token sizes. Your real TP/FP/FN come from diffing actual trajectories against a gold set, and your tools, their supports, and their stakes will differ. What is *not* illustrative is the structure: F1 is the harmonic mean of precision and recall and equals $\frac{2\mathrm{TP}}{2\mathrm{TP}+\mathrm{FP}+\mathrm{FN}}$, full stop; macro weights tools equally while micro weights calls equally; and whenever a rare class fails, micro will sit *above* macro and quietly hide it. Change the counts and the two averages move; that ordering and that hiding do not. Carry this into every eval dashboard: a single aggregate F1 is a summary, not a verdict — always keep the per-tool column next to it.

Appendix A — Reference implementation

Every number in this document is produced by the program below. It needs no libraries. Change the `tools` dictionary to score your own suite.

```python
# tool_f1.py -- reproduces every number in "Tool-Call F1, by Hand".
# Per-tool pooled counts over a small eval suite of the Refund Resolver.
tools = {
    "crm_lookup": dict(TP=18, FP=2, FN=0),
    "shipping_status": dict(TP=12, FP=3, FN=5),
    "refund": dict(TP=2, FP=4, FN=4),
}

def prf(TP, FP, FN):
    P = TP / (TP + FP) if (TP + FP) else 0.0
    R = TP / (TP + FN) if (TP + FN) else 0.0
    F = 2*P*R / (P + R) if (P + R) else 0.0 # harmonic mean
    Fa = 2*TP / (2*TP + FP + FN) if (2*TP+FP+FN) else 0.0 # identity, Eq.(2)
    return P, R, F, Fa

```

```python
# --- per-tool table ---
f1s = []
for name, c in tools.items():
    P, R, F, Fa = prf(**c)
    f1s.append(F)
    print(f"{name:16s} P={P:.4f} R={R:.4f} F1={F:.4f} (identity {Fa:.4f})")

# --- single-trajectory sanity check (the module's F1=0.86) ---
_, _, f_demo, _ = prf(TP=3, FP=1, FN=0)
print(f"module single-traj: F1={f_demo:.4f}") # 0.8571

# --- macro: mean of per-tool F1 ---
macro = sum(f1s) / len(f1s)
print(f"MACRO F1 = {macro:.4f}")

# --- micro: pool all counts, compute once ---
TP = sum(c["TP"] for c in tools.values())
FP = sum(c["FP"] for c in tools.values())
FN = sum(c["FN"] for c in tools.values())
Pm, Rm, Fm, _ = prf(TP, FP, FN)
print(f"MICRO pool TP={TP} FP={FP} FN={FN} P={Pm:.4f} R={Rm:.4f} F1={Fm:.4f}")
print(f"micro - macro = {Fm - macro:.4f}")

# Expected output:
# crm_lookup P=0.9000 R=1.0000 F1=0.9474 (identity 0.9474)
# shipping_status P=0.8000 R=0.7059 F1=0.7500 (identity 0.7500)
# refund P=0.3333 R=0.3333 F1=0.3333 (identity 0.3333)
# module single-traj: F1=0.8571
# MACRO F1 = 0.6769
# MICRO pool TP=32 FP=9 FN=9 P=0.7805 R=0.7805 F1=0.7805
# micro - macro = 0.1036
```

— end of document —